

Layouts y componentes con Blade

Blade

- Motor de plantillas oficial de Laravel
- Ventajas:
 - Ligero y rápido
 - Sintaxis similar a PHP pero más limpia
 - Integración sencilla con HTML
 - Agrega seguridad contra ataques XSS al imprimir variables
 - Permite la creación de componentes reutilizables

Comparación con PHP

Acción	Etiquetas PHP	Directivas de Blade
Imprimir variable	<?= \$var ?>	{{ \$var }}
Foreach	<?php foreach(\$items as \$item): ?> <?php endforeach; ?>	@foreach(\$items as \$item) @endforeach
Condición	<?php if(condicion): ?> <?php else: ?> <?php endif; ?>	@if(condicion) @else @endif
Apertura PHP	<?php código ?>	@php código @endphp

Documentación oficial de Blade: <https://laravel.com/docs/12.x/blade>

Vamos a rescribir la vista de transacciones con directivas de Blade

Componentes

- Son clase o archivos que encapsulan una parte reutilizable de la interfaz o la vista
- Ejemplos: botones, menús, mensajes de estado, formularios, etc.
- Ventajas:
 - Permiten definir un componente una única vez y luego utilizarlo en varias partes de la aplicación
 - Agregan un sentido de consistencia a la aplicación
 - El cambio visual o lógico en un componente se realiza en un solo lugar
 - La sintaxis de uso es similar a HTML

Creación de un componente

- Creamos el componente con el comando:
`php artisan make:component menu`
- Esto genera dos archivos:
 1. **resources/views/components/menu.blade.php**: esta es la vista del componente, donde agregamos el código HTML
 2. **app/View/Components/Menu.php**: esta clase es opcional, permite definir datos internos del componente.
- Editamos el archivo menu.blade.php y agregamos el código correspondiente.
- Luego lo importamos en la vista requerida:
`<x-menu />`

Creación del componente menu

1. Crear el componente de menú
2. Agregarlo a la vista **transacciones.blade.php**
3. Agregar estilos para el menú activo
 - Esto lo podemos lograr utilizando el método **request()->routeIs('nombreruta')**, el cual devuelve true si nos encontramos en la ruta definida

Creación de layouts

- **Layout:** plantilla base que define una estructura común a ser usada en más de una vista.
- Para aplicar layouts usamos las siguientes directivas de Blade:
 - **@extends('nombre')**: indica que una vista usa un layout base ya definido
 - **@section('nombre')**: define una sección dentro de la vista “hija”
 - **@yield('nombre')**: muestra una sección que ha sido definida en la vista “hija” dentro del layout
- Manos a la obra: crear un layout general para el proyecto.

Manejar estilos con CSS tradicional

- **Opción 1:** utilizar estilos CSS tradicionales
- En cualquier vista o componente podemos abrir una etiqueta `<style>` y definir los estilos CSS necesarios
- Ventajas:
 - Sintaxis conocida
 - Facilidad de implementación
 - Tiene buen soporte en navegadores
- Desventajas:
 - Se complica gestionar estilos de esta forma a medida que crece el proyecto
 - Posibilidad de crear estilos repetidos o que colisionen.

Manejar estilos con Tailwind CSS

- **Opción 2:** utilizar la librería Tailwind CSS para gestionar los estilos
- Librería que proporciona clases con estilos predefinidos (también permite personalizar estilos)
- Ventajas:
 - No hace falta escribir estilos adicionales a mano
 - Estandarización del diseño de las páginas
 - Buena adopción por parte de los sistemas de IA
 - Compilación automática a CSS mediante Vite
- Desventajas
 - Curva de aprendizaje media (hay que aprenderse las clases)
 - HTML verboso (se puede evitar usando correctamente los componentes)

Instalar Tailwind en Laravel

- Ejecutar el comando de instalación

```
npm install tailwindcss @tailwindcss/vite
```

- Configurar el archivo vite.config.js
- Importar Tailwind en resources/css/app.css
- Iniciar el proceso de compilación

```
npm run dev (no hace falta si usamos composer run dev)
```

- Usarlo en el proyecto

Documentación de instalación:

<https://tailwindcss.com/docs/installation/framework-guides/laravel/vite>